

Pending Template Updates: Comprehensive System Design

Overview & The Core Constraint My goal is to show users which of their engagement files have pending template updates and provide a readable summary of the changes. The architectural bottleneck I have to work around is that opening an engagement to check its version takes roughly 1 minute. I absolutely cannot make users wait for this during a page load.

Assumptions

- Template versions are cumulative (e.g., v7 includes v5 and v6).
- An existing login system identifies the user's firm.
- The system operates across a few geographic regions (e.g., US, EU, Canada).

Part 1: Target Architecture

I bypass the 1-minute loading penalty by using an event-driven architecture with three core ideas: a tracking table, a background worker, and reusable AI summaries.

As illustrated in **Figure 1 (Architecture: shared components and per-region components)**, the system splits responsibilities between a global template database and isolated regional environments.

The End-to-End Workflow:

1. **Publish:** The content team publishes a new template (e.g., v7), and the template database emits an event.
2. **Table Update:** Each region's updater service catches the event and adds v7 to its version list within seconds.
3. **Instant Query:** The API checks the tracking table and instantly flags any engagement older than v7 without opening the files.
4. **Summary Generation:** The system generates one AI summary per starting version (e.g., v4 to v7), which is ready in minutes.
5. **User Decision:** The user sees the summary, chooses to apply or decline, and the decision is saved and sent to the engagement system.

Handling Multiple Updates: If multiple updates accumulate, I offer the newest version combined with one summary from the engagement's current version. Users cannot skip intermediate versions. If a user declines an update (e.g., v5), the alert is hidden until a newer version (e.g., v7) arrives, which will then note that it includes previously declined changes.

Security & Data Isolation:

- **Firm Isolation:** I enforce this at the database level via PostgreSQL Row-Level Security (RLS). The database only returns rows for the specified firm, preventing cross-firm leaks even if my API code forgets a filter.
- **Access & Audit:** The API verifies user permissions before setting the firm. Decisions are saved in an add-only history table that the API cannot edit or delete.
- **Data Residency:** Each region has a dedicated database and API. Client data never leaves its region; only template versions and AI summaries are shared globally.

Part 2: Implementation Plan

I plan to roll this out with zero downtime, running alongside existing systems. Estimated effort is for 2-3 engineers.

- **Phase 1 - Foundations (2 weeks):** Build RLS tables, events, and updaters in shadow mode.
- **Phase 2 - Backfill (2 weeks to build, days to run):** The background worker checks existing engagements. At 20 concurrent calls, 100,000 engagements will take ~3.5 days to backfill. Until verified, files show as "Checking..."
- **Phase 3 - API & UI (2 weeks):** Build the pending list, decision API, and history views.
- **Phase 4 - Summaries (2 weeks):** Implement AI diff processing, automated checks, plain-text fallbacks, and locked storage.
- **Phase 5 - Rollout (2 weeks):** Enable for a pilot firm, then firm-by-firm as their files are verified.

Part 3: Testing Strategy

- **Worker Reliability:** I will verify concurrency limits, duplicate event handling, pause behavior on overload, and fair turns between firms.
- **Firm Isolation:** I will test PostgreSQL 16 RLS to ensure firms cannot read cross-firm rows and ensure the history table is immutable.
- **Logic & AI:** I will test multiple update rules, out-of-order events, and ensure AI failures gracefully degrade to plain text.
- **Load Testing:** I plan to inject worker crashes and simulate engagement system slowdowns to ensure resilience.

Part 4: Evaluation & Observability

- **Metrics:** I will track pending engagements per firm/product, the age of the oldest unknown file, and the worker's queue size and failure rates.
- **Auditability:** Authorized users can query the add-only decision history (who, when, what version, and what summary was shown).
- **Performance Targets:** My proposed targets are: new versions visible across regions in <60 seconds (99% of the time); summaries ready in <10 minutes; 99.5% of engagements known after backfill.
- **AI Quality & Defense:** Summaries are stored using S3 Object Lock alongside the prompt version and diff hash, ensuring I can defend a summary months later.
- **Cost:** I estimate ~\$500-800/month (PostgreSQL + containers across 3 regions). AI costs are minimal (~\$50/month) because summaries are reused.

Part 5: Failure Modes & Trade-offs

How I Handle Failures:

- *Lost/Duplicated Events:* Events carry timestamps (newer wins). The system cross-checks the version when a file is eventually opened; mismatches trigger a re-check.
- *Engagement System Overload:* My workers have a hard concurrency limit and will pause completely if told the system is "too busy".
- *AI Failures:* The system falls back to a plain-text code-generated change list.

- **Worker Crashes:** State lives in the database. A locked reservation simply expires and another worker picks it up.

Explicit Trade-offs I Made:

1. **Tracking Table vs. Real-Time:** I chose to accept *Eventual Consistency* (up to a minute delay) to achieve instant UI load times.
2. **Generic Summaries:** I use one summary per version pair. This protects client data and saves money, but means the AI cannot explain client-specific financial impacts.
3. **PostgreSQL RLS vs. DB-per-firm:** I chose RLS for simplicity and cost, though I acknowledge a DB-per-firm offers stronger physical isolation.

Where I am least sure:

1. **RLS vs. Database-per-firm:** While I chose RLS for cost and simplicity, a stricter customer could reasonably demand database-per-firm separation.
2. **Cumulative Versions:** I built this assuming versions are cumulative; if they are not, my logic for offers and summaries requires rework.
3. **The Numbers:** My SLAs (like the 60-second target), phase estimates, and costs are rough proposals that I would need to confirm against real measurements and the engagement team's actual capacity.

(Out of scope: applying updates, bulk decisions, notifications, multi-language summaries)

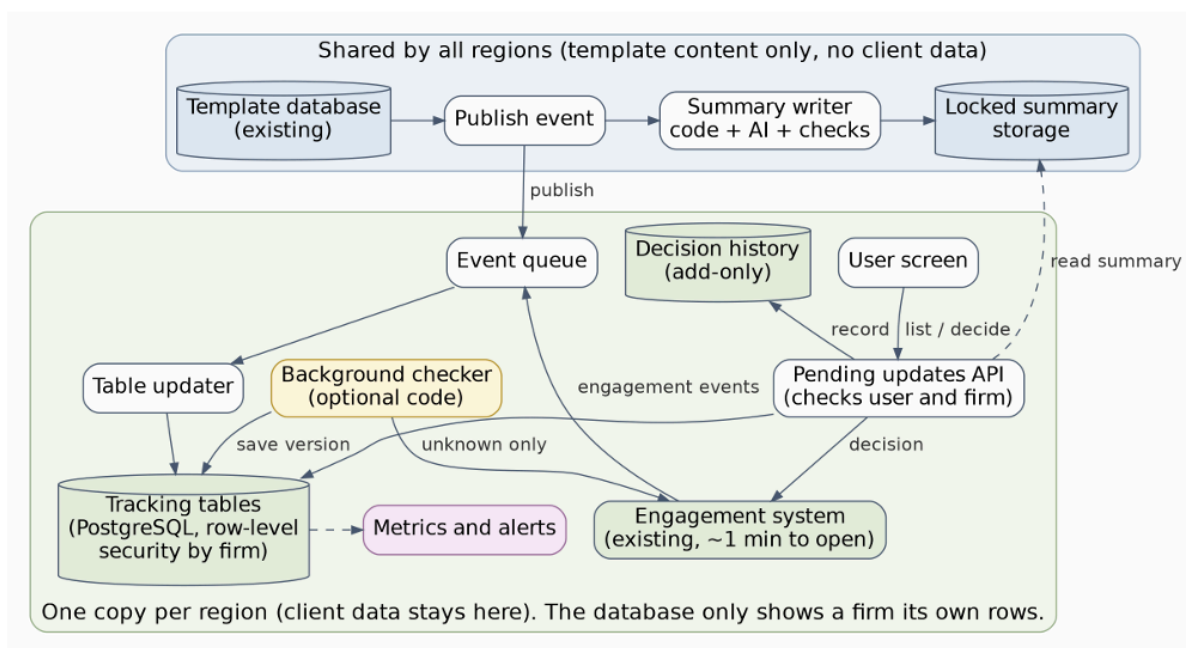


Figure 1. Architecture: shared components and per-region components